ESTRUTURA DE DADOS 1 · PROJETO 4

Catálogo de Produtos com Tags Automáticas

Um sistema que lê a descrição de um produto, descobre sozinho em que categoria ele se encaixa e o guarda de um jeito que torna a busca praticamente instantânea — mesmo com dezenas de milhares de produtos.

TABELA DE DISPERSÃO · ÍNDICE INVERTIDO · IA LOCAL

Documentação do sistema · decisões, medições e resultados

O que você vai encontrar aqui

SUMÁRIO

01	**O que é este sistema**	O problema e a solução
02	**Receber o produto**	Nome, descrição e a limpeza invisível
03	**A inteligência artificial**	As sete melhorias que precisei fazer
04	**Guardar na tabela hash**	O armário de gavetas numeradas
05	**Buscar pelas tags**	Virar a chave do avesso
06	**Os erros que cometi**	E o que eles mudaram no projeto
07	**Limites e como usar**	O que o sistema não faz

A IDEIA CENTRAL

Toda vez que medi em vez de assumir, o resultado foi diferente do que eu esperava — e **duas decisões do projeto mudaram por causa disso**. Este documento registra o caminho, não só o destino.

01 O que é este sistema

O PROBLEMA E A SOLUÇÃO

Imagine uma loja online com cem mil produtos. Alguém procura por *"tênis azul de corrida"*. Como o site encontra isso rápido?

O jeito óbvio seria abrir produto por produto, conferir se bate com o pedido e montar a lista. Funciona com dez produtos. Com cem mil, o cliente desiste antes de a página carregar.

As duas coisas que o sistema faz

PRIMEIRO · CATEGORIZAR SOZINHO

Ao cadastrar um **"Tênis azul para corrida em asfalto"**, o sistema lê o texto e decide por conta própria que aquilo é um calçado, é esportivo, serve para corrida e é azul. Essas palavras são as *tags*, e ninguém precisa digitá-las.

SEGUNDO · GUARDAR DE FORMA INTELIGENTE

Em vez de uma lista que precisa ser percorrida, usei uma estrutura em que encontrar algo **não depende de quantas coisas existem guardadas**. É a parte que a disciplina avalia, e onde está o esforço maior.

O caminho de um produto

1. **O texto é limpo**
   Minúsculas, sem acento, sem pontuação, sem palavras de ligação
2. **As palavras viram tags**
   calçado, esporte, azul, corrida, asfalto
3. **O produto é guardado**
   E cada tag vira um atalho que leva direto até ele

<0,1s	<0,001s	50 mil
para cadastrar um produto	para buscar	produtos no teste

02 Receber o nome e a descrição

PARTE 1 DO ENUNCIADO

O sistema recebe duas informações: um nome (**"Tênis azul"**) e uma descrição (**"para corrida em asfalto"**). A partir daí ele cuida de tudo: gera um código de identificação, cria as tags e guarda.

Por que separar nome e descrição

Poderia ser um texto só. Separei porque as duas coisas têm pesos diferentes: o nome diz o que o produto **é**, a descrição diz **como** ele é. As tags vindas do nome aparecem primeiro na lista, porque são as mais importantes para quem procura.

A parte invisível: arrumar o texto antes de usar

Parece um detalhe e é fundamental. As pessoas escrevem de formas diferentes:

```
"Tênis Azul"     "tenis azul"     "TÊNIS, AZUL!"
```

São a mesma coisa para uma pessoa e **três textos completamente diferentes** para um computador. Se guardasse do jeito que vem, quem cadastrasse com acento nunca seria encontrado por quem buscasse sem.

O QUE É FEITO	POR QUÊ
Vira minúsculo	"TÊNIS" e "tênis" passam a ser o mesmo
Perde os acentos	"tênis" vira "tenis"
Pontuação vira espaço	Para "azul,corrida" não virar "azulcorrida"
Palavras de ligação saem	"para", "de", "em" estão em toda descrição e não distinguem nada

> **O DETALHE QUE FAZ FUNCIONAR**
>
> A mesma limpeza é aplicada nos **dois lados**: ao cadastrar e ao buscar. É isso que faz alguém digitar `CALÇADO` e encontrar um produto guardado como `calcado`.

03 A inteligência artificial

PARTE 2 DO ENUNCIADO

Cada produto é analisado por um modelo de inteligência artificial que roda **dentro do próprio computador**. Ele lê o nome e a descrição e responde a que categoria o produto pertence.

> **RODA LOCAL, NÃO NA INTERNET**
>
> Não precisa de senha, não precisa de conexão, não custa nada por uso e nenhum dado sai da máquina. É o modelo do Apple Intelligence, já instalado em Macs recentes.

A IA de fábrica não serve

Ligar a IA foi fácil. Fazer com que ela entregasse algo aproveitável deu trabalho. **Foram sete melhorias**, e cada uma veio de um problema concreto que apareceu testando.

1 · Impedir que ela inventasse palavras

A primeira versão pedia: *"responda com as tags separadas por vírgula"*. Perguntei o mesmo três vezes e recebi três respostas diferentes.

```
[1]  calçados; tênis; corrida; asfalto; azul; esportivo
[2]  calçados, tênis, corrida, asfalto, azul, esportivos
[3]  calçados, esportivos, corrida, azul, tênis, esportividade
```

Uma delas inventou "esportividade", que não significa nada. Outra trocou a vírgula por ponto e vírgula. **Pedir educadamente no texto não garante nada.**

A solução foi mudar o tipo da resposta: em vez de texto livre, a IA passou a ter que **escolher de uma lista fechada**. Ela deixou de poder inventar, porque não existe onde escrever algo fora da lista.

ANTES

Dizia `calcado` para um produto e `calcados` para outro. O sistema tratava como categorias sem relação.

Cadastrei 6 calçados, busquei por calçado: **apareceram 4**.

DEPOIS

A lista fechada não permite variação. A mesma categoria sempre vira a mesma palavra.

Mesmo teste: **apareceram os 6**.

2 · Tirar a aleatoriedade

Modelos têm um ajuste que controla o quanto podem variar — é o que os faz parecer criativos. Num catálogo isso é defeito: se o mesmo produto pode receber categorias diferentes dependendo do dia, não dá nem para escrever um teste. Zerei esse ajuste.

3 · Fazer a pergunta certa

Aqui errei feio, e um teste barato me salvou.

Minha ideia era perguntar palavra por palavra. Antes de implementar, testei:

```
guardachuva  →  acessório     certo
cafeteira    →  cozinha       certo
dobravel     →  calçado       errado
resistente   →  calçado       errado
xyzabc       →  calçado       errado
```

O QUE TERIA ACONTECIDO

`xyzabc` não é palavra de idioma nenhum e virou calçado. Em treze palavras testadas, a IA usou a opção "isto não é um produto" **uma única vez**.

Se eu tivesse implementado assim, o sistema guardaria que "dobrável significa calçado" **para sempre**, e toda cadeira dobrável apareceria na busca por tênis.

O erro não era da IA, era da minha pergunta. "Dobrável" sozinho não quer dizer nada — nem para ela, nem para uma pessoa. Perguntando sobre a frase inteira, com contexto, ela acerta e sabe recusar:

```
Guarda-chuva dobrável automático  →  acessório
xyzabc qwerty                     →  desconhecida
```

4 · Deixar uma regra simples fazer o que ela fazia pior

Eu queria que a IA apontasse qual palavra nomeia o produto — "frigideira" em "Frigideira antiaderente de cerâmica". Pedi isso instruindo para copiar uma palavra só. **Ela devolveu a frase inteira em sete dos oito casos.**

A solução não foi insistir, foi perceber que aquilo nem precisava dela. Em português, nome de produto começa pelo substantivo: **Frigideira** antiaderente, **Esteira** ergométrica, **Trena** a laser. Escrevi uma regra de três linhas que pega a primeira palavra significativa.

1 de 8	16 de 16
acertos da IA	acertos da regra simples

VALE GUARDAR ESSA COMPARAÇÃO

Não é que a regra seja mais inteligente. É que ela aproveita uma regularidade que existe no problema — e a IA não tinha motivo para respeitar.

5 · Limpar o que ela devolve

Mesmo acertando a categoria, a IA entrega fora do padrão. Na saída real encontrei acentos onde não deveria (`calçado`), a mesma tag repetida duas vezes, tags com duas palavras (`roupa interior`) e até português de Portugal (`desporto` em vez de esporte).

> **A CONCLUSÃO QUE CONTRARIA A EXPECTATIVA**
>
> Usar inteligência artificial **não dispensou nenhuma** das decisões de estrutura de dados do projeto. Ela substituiu o vocabulário, não a disciplina de organizar.

6 · Não perguntar duas vezes a mesma coisa

Cada consulta leva cerca de meio segundo — lento demais para fazer sempre. Então o sistema anota o que aprendeu. E a melhoria que fez diferença foi anotar a **palavra**, não o produto:

```
"Frigideira antiaderente de cerâmica"  →  cozinha
          ↓
o sistema guarda:  frigideira = cozinha
          ↓
qualquer frigideira futura já sai classificada, sem consultar nada
```

15 de 15	**9s → 0,0002s**
produtos novos sem precisar da IA	tempo total desses produtos

Mas generalizar é perigoso. `Jogo de chaves de fenda` é ferramenta e `jogo de facas` é cozinha. Se o sistema decidisse que "jogo" é uma das duas, erraria a outra. Hoje ele percebe sozinho quando a mesma palavra aparece em categorias diferentes, **marca como ambígua e para de usá-la**.

7 · Saber quando não consultá-la, e quando não confiar nela

Duas descobertas tardias, as duas vindas de ver o programa rodando.

> **O CASO DO VIOLÃO**
>
> O sistema só chamava a IA quando o produto não gerava tag nenhuma. Um *"Violão acústico com cordas de nylon"* recebia "nylon" e "material", o sistema achava que já sabia o suficiente, e a IA nunca era consultada.
>
> Medindo, o problema era maior: **quatro em cada dez produtos** estavam assim. Uma cadeira de couro não era móvel; um tapete não era decoração. O critério confundia *"ter alguma tag"* com *"saber o que o produto é"*.

A segunda: tentei medir o quanto a IA estava segura, perguntando várias vezes e vendo se mudava de ideia. A ideia era bonita, os números foram modestos — detectou três dos sete erros conhecidos, deixou quatro passar e ainda acusou uma resposta certa. Descobri que **estabilidade não é o mesmo que acerto**: a IA pode estar consistentemente errada.

O que ela continua fazendo mal

Uma limitação não teve conserto: **a IA não sabe dizer "não sei"**.

> **A CAMISA QUE FICOU AZUL**
>
> Perguntei a cor de uma *"Camisa de algodão"* — texto que não menciona cor nenhuma. Ela respondeu **azul**. Inventou. Tentei deixar o campo opcional e instruir explicitamente para só responder se a cor estivesse escrita: preencheu igual.
>
> Num catálogo isso é grave: o cliente filtra por azul e recebe uma camisa branca.

A divisão de trabalho que resultou

QUEM DECIDE	O QUÊ	POR QUÊ
A IA	A categoria do produto	Acerta até em produtos que ninguém cadastrou antes
O vocabulário	Cor, material, uso	Só informa o que está escrito no texto, nunca inventa

O vocabulário é uma lista que escrevi, ligando palavras a tags: "azul" é uma cor, "couro" é um material, "corrida" é um uso. **É burro, mas é exato.** Cada um cuida do que faz bem.

O resultado final

Depois das sete melhorias, a IA deixou de ser um gerador imprevisível e virou uma peça confiável:

- responde sempre a mesma coisa para a mesma entrada;
- não consegue inventar uma categoria que não existe;
- sabe recusar quando não reconhece o produto;
- é consultada uma vez por palavra nova, e nunca mais;
- o que ela ensina fica num arquivo de texto comum, que uma pessoa pode ler, conferir e corrigir à mão.

> **O ARGUMENTO QUE MAIS ME CONVENCEU**
>
> Revisando o que a IA tinha classificado, achei sete erros — ela colocou uma trena a laser em artigos de esporte e um colchão em acessórios. Corrigi os sete editando um arquivo de texto, **em um minuto**.
>
> Se esse conhecimento estivesse dentro de um modelo treinado, não haveria o que revisar nem como corrigir um caso específico.

04 Guardar na tabela de dispersão

PARTE 3 DO ENUNCIADO

A IDEIA, EM TERMOS SIMPLES

Imagine um armário com mil gavetas numeradas e mil papéis para guardar. Se guardar na ordem que chegam, achar um papel depois exige abrir uma gaveta por vez.

A tabela de dispersão faz diferente: ela **faz uma conta com o conteúdo do papel**, e essa conta diz em qual gaveta ele deve ficar. Para achar depois, basta refazer a conta. Você vai direto à gaveta certa, sem abrir nenhuma outra.

Não importa se o armário tem mil ou um milhão de gavetas: **o tempo é o mesmo**.

Por que não usei o que a linguagem já oferece

O Python já vem com uma estrutura pronta que faz exatamente isso. Usá-la seria uma linha de código. **Não usei de propósito** — a disciplina é sobre entender a estrutura, e a versão pronta esconde justamente o que está sendo estudado.

Há ainda um motivo prático: a função pronta do Python muda de resultado a cada execução, por segurança da linguagem. Isso tornaria impossível demonstrar em sala que determinado produto vai para determinada gaveta.

Quando dois produtos querem a mesma gaveta

Isso tem nome — **colisão** — e não é falha: é inevitável. Existem infinitos textos possíveis e um número limitado de gavetas.

A pergunta certa não é como evitar, é o que fazer quando acontece. Existem duas respostas, e eu testei as duas.

A QUE ESCOLHI

Cada gaveta guarda uma **corrente**. O segundo produto é pendurado no primeiro, e a busca percorre a correntinha — normalmente de um ou dois itens.

A OUTRA

O segundo produto **procura a próxima gaveta livre**, e a busca precisa seguir o mesmo caminho para encontrá-lo.

Implementei as duas e medi. Com o armário quase cheio:

1,3	62	48×
comparações por busca na estratégia escolhida	comparações por busca na outra	de diferença

O motivo é que, na segunda, um produto deslocado ocupa a gaveta do vizinho, que desloca o próximo, e forma-se **um bloco que cresce sozinho**. Na primeira, uma corrente longa não atrapalha as gavetas vizinhas.

Como o sistema cresce sozinho

Quanto mais cheio o armário, mais longas as correntes e mais lenta a busca. Por isso o sistema vigia a própria ocupação: **quando passa de 75%, ele dobra de tamanho e redistribui tudo**.

POR QUE REDISTRIBUIR É OBRIGATÓRIO

A gaveta de cada item depende do número total de gavetas. Mudando esse número, muda a conta. Se eu copiasse os itens para as mesmas posições em um armário maior, eles continuariam lá, mas **a busca passaria a procurar no lugar errado** e não acharia nada.

É um erro traiçoeiro porque tudo parece funcionar — só a busca falha.

Um detalhe que rendeu uma surpresa

A quantidade de gavetas é sempre um número primo. A teoria diz que isso espalha melhor os itens, e eu confirmei medindo — com uma diferença maior do que esperava.

QUANTIDADE DE GAVETAS	GAVETAS USADAS	GAVETAS VAZIAS	MAIOR CORRENTE
1024 — potência de dois	**708**	**316**	**21**
1031 — primo	**1031**	**0**	**12**

O RESULTADO

Com 1024 gavetas, **316 nunca receberam nada**. Quase um terço do armário ficou vazio enquanto o resto se amontoava. Com 1031, nenhuma sobrou vazia.

05 Buscar pelas tags

PARTE 4 DO ENUNCIADO

A armadilha em que quase caí

Guardar o produto na tabela resolve *"me mostre o produto P007"*. Mas o enunciado pede outra coisa: achar produtos **pelas tags**.

> **O ERRO QUE QUASE COMETI**
>
> Com uma tabela só, buscar por tag significaria abrir cada produto e conferir as tags de cada um — ou seja, voltar a percorrer o catálogo inteiro. Exatamente o problema que a estrutura deveria resolver.
>
> **A estrutura estaria certa, respondendo à pergunta errada.**

A solução: virar a chave do avesso

Criei uma segunda tabela, construída ao contrário. Na primeira, o produto é a chave. Na segunda, a **tag** é a chave.

```
tabela de produtos              tabela de tags

P001 → Tênis azul               calçado → P001, P003
P002 → Camisa azul              azul    → P001, P002
P003 → Chuteira preta           corrida → P001
```

Agora buscar por "azul" é uma consulta direta que já devolve a resposta pronta. É a mesma ideia do **índice remissivo** no fim de um livro: em vez de ler o livro inteiro procurando uma palavra, você olha a palavra no índice e ela te diz as páginas.

45×	50 mil	cresce
mais rápido que percorrer o catálogo	produtos no teste	quanto maior a loja, maior a vantagem

O preço, e por que vale

A mesma informação fica em dois lugares. Isso gasta mais memória e cria uma obrigação: ao remover um produto, é preciso limpar os dois. Esquecer o segundo deixa a tag apontando para um produto que não existe mais.

Vale a pena porque, numa loja, **busca acontece milhares de vezes para cada cadastro**. Gastar um pouco na hora de guardar para economizar muito na hora de procurar é uma troca óbvia.

Uma terceira tabela, para o que as tags não cobrem

Percebi um problema testando: produtos classificados pela IA recebiam uma única tag. Uma *"Frigideira antiaderente de cerâmica"* ficava só com "cozinha", e quem procurasse **cerâmica** não achava — apesar de a palavra estar escrita ali.

Criei então um terceiro índice, que guarda as palavras literalmente escritas no produto. Os dois convivem e fazem coisas diferentes:

BUSCA POR	QUEM ENCONTRA	O QUE ISSO EXIGE
`calçado`	**índice de tags**	Entender que tênis é um calçado — a palavra não está escrita no produto
`cerâmica`	**índice de texto**	Apenas que a palavra esteja no texto — não é categoria de nada
`corrida`	**os dois**	Está escrita no produto e também é uma tag

> **POR QUE OS DOIS PRECISAM EXISTIR**
>
> **Nenhum dos dois encontra o que o outro encontra.** Um busca por significado, o outro por palavra escrita. São estratégias complementares, não alternativas.

As três tabelas do sistema

1. **Produtos por código**
   P001 leva direto ao produto — acesso instantâneo
2. **Tags por significado**
   calçado leva à lista de todos os calçados
3. **Palavras escritas**
   cerâmica leva a todo produto que menciona cerâmica

06 Os erros que cometi

E O QUE ELES MUDARAM NO PROJETO

Esta seção existe porque, para mim, foi a parte mais valiosa do trabalho. Em vários momentos escrevi uma afirmação que parecia óbvia, fui medir, e o resultado foi diferente.

Três vezes em que a medição mudou uma decisão

ERRO 1 · REPETI TEORIA SEM VERIFICAR

Escrevi que a estratégia de colisão que descartei *"piora com muitas remoções"*. Era o que eu tinha lido. Implementei a alternativa e fiz vinte mil ciclos de remoção para comprovar: **o desempenho não piorou nada**.

O motivo real da desvantagem era outro — os blocos que crescem sozinhos. Corrigi a documentação deixando registrado o que eu havia escrito antes.

ERRO 2 · ACHEI QUE UMA ESCOLHA IMPORTAVA

Achava que o limite de ocupação da tabela fazia diferença. Testei com 50% e com 75%, esperando ver distância, e os resultados vieram **idênticos** — porque a tabela dobra de tamanho ao crescer, e a ocupação acaba oscilando na mesma faixa nos dois casos.

ERRO 3 · UMA IDEIA BONITA QUE NÃO FUNCIONOU

Tentei medir o quanto a IA estava segura de cada resposta, perguntando várias vezes. Na prática detectou só **três dos sete erros** que eu já conhecia, e ainda acusou uma resposta certa. Mantive o recurso, mas serve para decidir o que revisar primeiro — não para julgar o que está certo.

E os defeitos que só apareceram usando o programa

O QUE ACONTECEU	POR QUÊ
Uma chuteira preta saía sem tag de cor	A regra de plural não trata gênero, e "preta" não estava no vocabulário
A palavra "de" virou tag	Eu filtrava as palavras inúteis antes de separar as compostas, e "de homem" passava inteira
Um violão foi classificado só pelo material das cordas	O sistema achava que já tinha tags suficientes e nem consultava a IA

DE ONDE VÊM OS TESTES DO PROJETO

Cada um desses defeitos virou uma verificação automática. É daí que vem a maior parte dos **123 testes**: não de casos que imaginei, mas de erros que cometi.

07 Limites e como usar

ONDE O SISTEMA TERMINA

Listar limitações não é modéstia — é saber onde o sistema termina.

LIMITAÇÃO	O QUE SIGNIFICA
Os dados somem ao fechar	O catálogo vive na memória. Salvar em arquivo é assunto de outra disciplina.
O vocabulário é limitado	Palavras fora dele não geram tags de cor ou material. A IA cobre a categoria, não os atributos.
As categorias são uma lista fechada	Um violão vira "áudio", porque não existe categoria para instrumento musical. A lista é fechada de propósito — é o que impede a IA de inventar.
A IA precisa de um Mac recente	Sem ela, o sistema funciona com o vocabulário e com tudo que já foi aprendido, porque isso está gravado em arquivo.
O sistema não entende frases	Reconhece palavras. "Mochila para notebook" recebe a marcação de informática mesmo não sendo um equipamento.

Como usar

Não precisa de instalação nem de bibliotecas externas. Só do Python.

```
python3 main.py        # só com o vocabulário, funciona em qualquer computador
python3 main.py --ia   # com a inteligência artificial analisando cada produto
```

O programa abre um menu com as opções de cadastrar, listar, buscar por tag, buscar por várias tags ao mesmo tempo, buscar por palavra escrita e ver o estado interno das tabelas.

Para conferir que tudo funciona

```
python3 -m unittest discover -s testes -t .
```

São **123 verificações automáticas**. A maior parte nasceu de um defeito real encontrado durante o desenvolvimento, e existe para impedir que ele volte.

Para ver as medições citadas aqui

```
python3 benchmark.py           # busca com índice contra percorrer tudo
python3 benchmark_colisoes.py  # as duas estratégias de colisão
```

> **PARA FECHAR**
>
> Todo número citado neste documento sai de um desses dois comandos. Nenhum deles é estimativa — **todos podem ser conferidos rodando**.